

# 中国矿业大学计算机学院

## 2023 级本科生课程报告

课程名称： 数据挖掘基础

报告题目： 聚类算法原理与应用

报告时间： 2025.11.30

学生姓名： 王志豪

学 号： 08232337

专 业： 数据科学与大数据技术

任课教师： 祝汉城

课程报告成绩评定及评分依据

| 序号                                                 | 评分依据                                                  | 分值 | 实际得分 |
|----------------------------------------------------|-------------------------------------------------------|----|------|
| 1                                                  | 课程报告描述了数据挖掘技术或算法的基本原理                                 | 20 |      |
| 2                                                  | 课程报告详细描述了待解决的问题，如有可能参考教材给出解决问题的过程                     | 30 |      |
| 3                                                  | 课程报告使用 Python 程序设计语言（附源代码）对待解决问题进行了实验，给出了相关实验结果或可视化结果 | 40 |      |
| 4                                                  | 课程报告给出了个人学习体会                                         | 10 |      |
| 总分                                                 |                                                       |    |      |
| <div>教师评语：</div> <div>教师签名：</div> <div>年 月 日</div> |                                                       |    |      |

# 目 录

|                      |    |
|----------------------|----|
| 1 绪论 .....           | 1  |
| 2 问题描述 .....         | 1  |
| 3 聚类算法原理与优缺点分析 ..... | 2  |
| 3.1 基于划分的方法 .....    | 2  |
| 3.2 基于层次结构的方法 .....  | 2  |
| 3.3 基于密度的方法 .....    | 3  |
| 3.4 基于图/谱的方法 .....   | 3  |
| 3.5 基于模型的方法 .....    | 3  |
| 4 源代码及实现结果 .....     | 4  |
| 5 个人体会 .....         | 10 |

## 1. 绪论

聚类分析作为无监督学习中的核心技术，其目标在于依据样本间的相似性结构，对数据进行自动划分，以揭示潜在的模式与类别关系。在长期的发展过程中，聚类算法逐渐形成了多种具有代表性的理论体系，包括以优化目标函数为基础的划分类算法、以树状结构构建为核心的层次类算法、依赖密度分布识别任意形状簇的密度类算法、使用概率模型刻画数据生成过程的模型类算法，以及通过图论与谱分解刻画数据结构的谱类算法。不同类型算法在簇形状假设、噪声鲁棒性、参数敏感性以及对数据结构的适应性方面具有显著差异，因此需要在统一的数据环境下进行系统性评估与横向对比。为此，本实验选用结构清晰且广泛应用于教学与算法验证的鸢尾花(Iris)数据集作为研究对象，该数据集包含 150 条样本与四个连续特征，天然呈现出一个可分性强、但在部分类别间存在重叠的典型小规模数据结构，能够有效检验不同聚类算法对特征分布、簇形状与类间边界的敏感性。本研究从五类聚类算法体系中选取具有代表性的典型算法，包括基于划分的 K-Means、基于层次的凝聚层次聚类、基于密度的 DBSCAN、基于模型的高斯混合模型(GMM)以及基于谱方法的谱聚类，并在不对数据进行标准化处理的条件下，对这些算法在鸢尾花数据集上的聚类性能进行横向对比分析，旨在全面评估不同理论范式下的聚类方法在统一任务场景中的表现差异，并为实际应用中聚类算法的选择提供参考依据。

## 2. 问题描述

在本实验中，我们选用经典的鸢尾花(Iris)数据集作为研究对象，该数据集由 150 条样本组成，包含花萼长度、花萼宽度、花瓣长度和花瓣宽度四个连续型特征，分别对应三种鸢尾花类别。其数据结构具有两个显著特点：一是**特征维度低**且易于可视化，适合用于聚类算法的基础验证；二是其类别分布呈现出一定的**可分性**，其中山鸢尾(Setosa)与另外两类之间差异显著，而变色鸢尾(Versicolor)与维吉尼亚鸢尾(Virginica)在特征空间中存在**部分重叠**，为聚类任务提供了具有挑战性的结构特征。基于此，为全面评估不同聚类范式在面对同一数据集时的表现，本实验从主要聚类方法体系中各选取一个具有代表性的典型算法，包括划分式的 K-Means、模型式的高斯混合模型(GMM)、层次式的凝聚层次聚类(Agglomerative Clustering)以及密度式的 DBSCAN，并在不进行数据标准化的条件下对其进行统一实验。各算法分别对应不同的建模假设与

结构表达能力：K-Means 假设簇为凸状球形结构，强调最小化簇内误差；GMM 通过高斯分布的概率建模能够描述椭圆形及重叠区域；层次式方法通过逐步合并样本揭示层次结构；DBSCAN 则依赖密度连接性识别任意形状簇并具备噪声检测能力。通过对这些多范式算法在鸢尾花数据集上的聚类结果进行量化评测（包括 ARI、AMI、Homogeneity、Completeness、V-measure、Silhouette 等指标）以及二维可视化展示，本实验旨在系统分析不同聚类算法在相同数据条件下的性能差异、结构适配性与适用场景，从而为聚类模型的选择与理解提供参考依据。

### 3. 聚类算法原理与优缺点分析

#### 3.1 基于划分的方法

基于划分的方法的核心思想通俗来说就是：直接将样本划分成  $K$  个簇，依靠迭代优化目标函数实现。该部分将以 K-Means 算法为例，简述基于划分的聚类算法的原理以及优缺点。

K-Means 属于典型的划分式聚类算法，其目标是将数据划分为  $K$  个簇，通过迭代优化簇内平方误差（Within-Cluster Sum of Squares, WCSS）。算法流程包括随机初始化  $K$  个中心点、基于欧氏距离将样本分配到最近中心、随后更新中心为簇内样本均值并重复上述步骤直至中心收敛。其隐含假设包括簇为凸形、类间边界近似线性，适用于球状或紧密分布的数据。

K-Means 的优点在于算法实现简单、计算效率高、可扩展性强，适用于中大规模数据，并且在簇结构呈现接近球状、类间分布分离较好的情况下能够取得稳定效果；其缺点是需要预先指定簇数，对初始中心敏感，容易陷入局部最优，同时假设簇为凸形结构，因此对非球状、不同密度、存在噪声或离群点的数据适应性较差。

#### 3.2 基于层次结构的方法

基于层次结构的方法的核心思想通俗来说就是：通过建立层次树（dendrogram）来描述聚类过程。这里我们用 Agglomerative 为例来简析该类算法的原理与优缺点。

凝聚层次聚类是一种自底向上的层次式算法，起始时将每个样本视为独立簇，随后按簇间相似度（或距离）逐步合并，最终形成树状的层次聚类结构（树状图）。关键在于选择合适的链接方式，如单链接（最小距离）、完全链接（最大距离）、平均链接或 ward 链接（最小化类间平方误差增加）。它不需要事先设定簇数，而是在树状图上根据距离阈值进行切割。

凝聚层次聚类的优点是无需预先指定簇数，能够生成反映数据层次关系的树状结构，并且对不同形状的簇具有一定适应性；但其缺点是计算复杂度较高，不适合大规模数据，一旦合并操作执行便不可回退，因此对噪声、距离度量及链接方式较为敏感，并且在样本量增大后易出现层次结构不稳定的问题。

### 3.3 基于密度的方法

基于密度的方法的核心思想简单来说就是：簇为任意形状的密度相连区域。这里用来阐述该类方法的算法是 DBSCAN。

DBSCAN 是一种基于密度的聚类方法，通过密度可达性识别高密度区域为簇，同时自动识别噪声点。算法通过两个核心参数：邻域半径  $\epsilon$  和最低点数  $\text{MinPts}$  来定义核心点、边界点和噪声点；具有足够邻域密度的点作为核心点，其密度可达区域被认为属于同一簇。该算法能够发现任意形状的簇，并对噪声与离群点具有较强鲁棒性。

DBSCAN 的优点在于能够发现任意形状的簇，不依赖簇数预设，并且能够有效识别噪声点和离群点；然而其缺点是对参数  $\epsilon$  与  $\text{MinPts}$  较为敏感，在不同密度的簇结构中难以找到统一的最优参数，导致表现不稳定，同时在高维空间中由于距离退化问题，密度估计效果明显下降，从而影响聚类质量。

### 3.4 基于图/谱的方法

基于图/谱的方法的核心思想简单来说就是：将空间划分为多个网格单元，然后对密集网格聚类。这里用来阐释这类算法的是 Spectral Clustering。

谱聚类基于图论思想，将数据映射为相似度图，再通过图拉普拉斯矩阵的特征向量实现降维和聚类。其核心流程包括构建样本之间的相似度矩阵（如基于 RBF 核），计算图拉普拉斯矩阵的前  $K$  个特征向量以获取低维嵌入表示，再对嵌入后的表示应用 K-Means 进行最终聚类。其优势在于可识别复杂的非凸形簇，并能利用全局图结构进行聚类。

谱聚类的优点在于利用图拉普拉斯矩阵的特征向量刻画全局数据结构，能够识别复杂的非凸形簇，适合处理接近流形结构的数据，并在中小规模数据集上具有优秀表现；其缺点是需要构建相似度矩阵，时间与空间复杂度较高，不适用于大规模数据，并且相似度矩阵构建方式、尺度参数以及图结构选择都会显著影响最终结果，导致一定的不稳定性。

### 3.5 基于模型的方法

基于模型的方法的核心思想通俗来说就是：假定数据由某种概率模型生成，通过估计模型参数实现聚类。我们选取具有代表性的 GMM 算法来阐释该类算法。

高斯混合模型属于基于概率模型的软聚类方法，将数据视为由多个高斯分布混合生成，通过最大化似然估计拟合每个簇对应的高斯分布参数（均值、协方差及混合系数）。其优化通常采用 EM 算法：E 步计算每个样本属于各高斯成分的后验概率，M 步更新模型参数。相比 K-Means，GMM 能够建模椭圆形簇、重叠区域以及不同方差结构，因此表达能力更强。

GMM 的优点是以概率方式描述簇结构，能够刻画不同方差、不同形状（尤其是椭圆形）的簇，并通过软聚类提供更细粒度的样本归属信息，通常比 K-Means 更灵活且表达能力更强；其缺点包括容易陷入局部最优，对初始化敏感，对协方差矩阵估计要求较高，在样本量较小或簇结构高度重叠时可能产生不稳定解，同时其计算成本高于 K-Means。

## 4. 源代码及实现结果

本实验的所有关键代码按类别罗列如下所示：

### A. 导入关键 Python 包

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN, SpectralClustering
from sklearn.mixture import GaussianMixture
from sklearn.decomposition import PCA
from sklearn.metrics import (
    adjusted_mutual_info_score,      # 调整互信息
    homogeneity_score,              # 同质性评分
    completeness_score,            # 完整性评分
    v_measure_score,                # V-measure评分（同质性和完整性的调和平均）
    adjusted_rand_score,            # 调整兰德指数
    normalized_mutual_info_score,    # 规范化互信息
    silhouette_score                 # 轮廓系数
)
```

### B. 加载数据：

```
# =====
# 数据加载
# =====
# 加载鸢尾花数据集（包含150个样本，4个特征）
data = load_iris()
X = data.data                      # 特征矩阵，形状为(150, 4)
y_true = data.target               # 真实标签，形状为(150,)，包含3类（0,1,2）

# =====
# 评测函数 - 使用多种指标评估聚类结果
# =====
```

## C. 评测函数

```
# =====
# 评测函数 - 使用多种指标评估聚类结果
# =====
def evaluate(name, y_true, y_pred, X):
    """
    评估聚类算法的性能

    参数:
    -----
    name : str
        聚类算法的名称
    y_true : array-like
        真实的类标签
    y_pred : array-like
        算法预测的聚类标签
    X : array-like
        原始特征矩阵, 用于计算轮廓系数
    """
    print(f"\n=== {name} 聚类评测结果 ===")

    # AMI (调整互信息): 衡量真实标签和预测标签的一致性
    # 取值范围[-1, 1], 值越大越好, 0表示随机聚类
    print("AMI [调整互信息]:", adjusted_mutual_info_score(y_true, y_pred))

    # Homogeneity (同质性): 衡量同一聚类中的样本是否都来自同一真实类
    # 取值范围[0, 1], 值越大越好
    print("Homogeneity [同质性]:", homogeneity_score(y_true, y_pred))

    # Completeness (完整性): 衡量同一真实类的样本是否都被分配到同一聚类
    # 取值范围[0, 1], 值越大越好
    print("Completeness [完整性]:", completeness_score(y_true, y_pred))

    # V-measure: 同质性和完整性的加权调和平均
    # 取值范围[0, 1], 综合评估聚类效果
    print("V-measure:", v_measure_score(y_true, y_pred))

    # Adjusted Rand Index (ARI): 衡量两个聚类结果的相似度
    # 取值范围[-1, 1], 值越大越好, 0表示随机聚类
    print("Adjusted Rand Index (ARI):", adjusted_rand_score(y_true, y_pred))

    # Normalized Mutual Information (NMI): 规范化互信息
    # 取值范围[0, 1], 值越大越好
    print("Normalized Mutual Information (NMI):", normalized_mutual_info_score(y_true, y_pred))

    # Silhouette Score (轮廓系数): 衡量样本与其所属聚类的匹配程度
    # 取值范围[-1, 1], 值越接近1越好
    # 前提: 至少有2个聚类且不存在噪声点 (标签为-1)
    unique_labels = set(y_pred)
    if len(unique_labels) > 1 and -1 not in unique_labels:
        print("Silhouette Score:", silhouette_score(X, y_pred))
    else:
        print("Silhouette Score: 无法计算 (簇数不足或存在噪声点)")
```

## D. 可视化函数



```

# =====
# 可视化函数 - 使用PCA降维将聚类结果显示在2D平面上
# =====
def visualize(name, X, labels):
    """
    使用PCA降维将高维聚类结果可视化2D散点图

    参数:
    -----
    name : str
        聚类算法的名称
    X : array-like
        原始特征矩阵
    labels : array-like
        聚类标签
    """
    # 使用PCA降维到2个主成分，保留最多的数据信息
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X)

    # 创建图表
    plt.figure(figsize=(6, 5))

    # 绘制散点图，使用聚类标签为点着色
    plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap="viridis", s=45)

    plt.title(f"{name} 聚类结果 (PCA 2D)")
    plt.xlabel("PC1 (第一主成分)")
    plt.ylabel("PC2 (第二主成分)")
    plt.show()

```

## E. K-Means

我们从 `sklearn.cluster` 中导入 `KMeans` 类，设定聚类数以及随机种子，将待聚类的源数据带入 `kmeans` 的预测函数中得到对应标签，最后评估聚类效果并可视化聚类结果。

```

# =====
# 1. K-Means 聚类（基于划分的算法）
# =====
# K-Means 原理:
# - 将数据划分为K个不相交的聚类
# - 每个聚类由一个质心代表
# - 通过最小化样本到质心的距离平方和进行优化

kmeans = KMeans(
    n_clusters=3,      # 设置聚类数为3（与鸢尾花的3个品种相匹配）
    random_state=42    # 设置随机种子，确保结果可复现
)

# 拟合数据并预测聚类标签
kmeans_labels = kmeans.fit_predict(X)

# 评估聚类效果
evaluate("K-Means", y_true, kmeans_labels, X)

# 可视化聚类结果
visualize("K-Means", X, kmeans_labels)

```

## F. 层次聚类

过程同上述 K-Means，先导入 AC 算法，并设定聚类数，预测得到标签后评价最终结果。

```

# =====
# 2. 层次聚类 - Agglomerative (基于层次的算法)
# =====
# Agglomerative 聚类 原理:
# - 自下而上的聚合方式
# - 开始时每个样本为一个聚类, 逐步合并最近的聚类
# - 直到获得指定数量的聚类为止

hier = AgglomerativeClustering(
    n_clusters=3          # 设置最终的聚类数为3
)

# 拟合数据并预测聚类标签
hier_labels = hier.fit_predict(X)

# 评估聚类效果
evaluate("Agglomerative Clustering", y_true, hier_labels, X)

# 可视化聚类结果
visualize("Agglomerative Clustering", X, hier_labels)

```

#### G. DBSCAN

过程同上述 K-Means, 不过这里设定的参数是 `eps` 和核心点的最小邻域样本数, 分别为 0.4 和 4, 聚类得到标签后进行评测和可视化。

```

# =====
# 3. DBSCAN 聚类 (基于密度的算法)
# =====
# DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 原理:
# - 基于点的密度进行聚类
# - 核心点: eps邻域内样本数 >= min_samples
# - 边界点: 在核心点的eps邻域内, 但自己不是核心点
# - 噪声点: 既不是核心点也不是边界点 (标签为-1)
# 优点: 可以发现任意形状的聚类, 无需提前指定聚类数

dbscan = DBSCAN(
    eps=0.6,             # 邻域半径: 两点间距离小于此值时认为相邻
    min_samples=4        # 核心点的最小邻域样本数
)

# 拟合数据并预测聚类标签
db_labels = dbscan.fit_predict(X)

# 评估聚类效果
evaluate("DBSCAN", y_true, db_labels, X)

# 可视化聚类结果
visualize("DBSCAN", X, db_labels)

```

#### H. GMM

过程同 K-Means, 不过这里的 GMM 包是来自于 `sklearn.mixture`, 同样设定参数高斯分布数量、随机种子 3 和 42, 拟合完数据后, 将标签结果进行评测和可视化。

```

# =====
# 4. GMM 聚类（基于模型的算法）
# =====
# GMM (Gaussian Mixture Model) 原理:
# - 假设数据由K个高斯分布的混合生成
# - 通过EM算法估计每个高斯分布的参数（均值、协方差）
# - 每个样本被分配给概率最大的高斯分布
# 优点：提供了概率化的聚类结果，而不仅仅是硬分配

gmm = GaussianMixture(
    n_components=3,      # 设置高斯分布的数量为3
    random_state=42      # 设置随机种子，确保结果可复现
)

# 拟合数据并预测聚类标签
gmm_labels = gmm.fit_predict(X)

# 评估聚类效果
evaluate("GMM", y_true, gmm_labels, X)

# 可视化聚类结果
visualize("GMM", X, gmm_labels)

```

## I. 谱聚类

过程同 K-Means，设定聚类数，使用最近邻图构建相似度矩阵并设定随机种子，预测完标签后进评测和结果可视化。

```

# =====
# 5. 谱聚类（基于图/谱的算法）
# =====
# Spectral Clustering 原理:
# - 将数据看作图的顶点，样本相似度作为边权重
# - 使用图的拉普拉斯矩阵的特征向量进行降维
# - 在降维空间中使用K-Means进行聚类
# 优点：可以处理非凸形状的聚类，基于相似度而不是距离

spectral = SpectralClustering(
    n_clusters=3,          # 设置聚类数为3
    affinity='nearest_neighbors', # 使用最近邻图构建相似度矩阵
    random_state=42        # 设置随机种子，确保结果可复现
)

# 拟合数据并预测聚类标签
sp_labels = spectral.fit_predict(X)

# 评估聚类效果
evaluate("Spectral Clustering", y_true, sp_labels, X)

# 可视化聚类结果
visualize("Spectral Clustering", X, sp_labels)

```

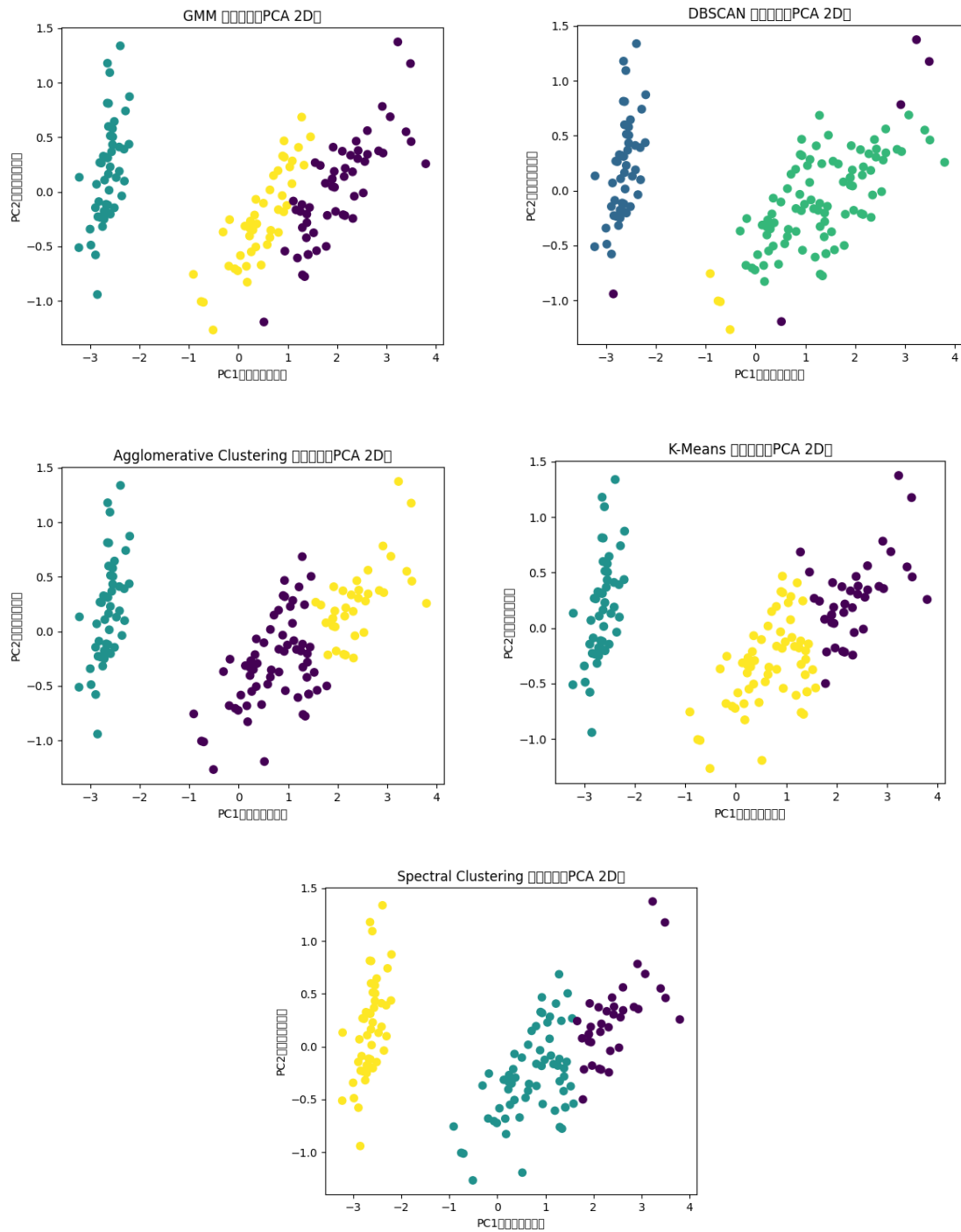
本次实验所用的评测指标分别为 AMI（调整互信息）、Homogeneity（同质性）、Completeness（完整性）、V-measure、Adjusted Rand Index (ARI)、Normalized Mutual Information (NMI)、Silhouette Score。

| 算法                  | AMI    | Homogeneity | Completeness | V-measure | ARI    | NMI    | Silhouette |
|---------------------|--------|-------------|--------------|-----------|--------|--------|------------|
| K-Means             | 0.7387 | 0.7364      | 0.7475       | 0.7419    | 0.7163 | 0.7419 | 0.5512     |
| Agglomerative       | 0.7672 | 0.7608      | 0.7796       | 0.7701    | 0.7312 | 0.7701 | 0.5543     |
| DBSCAN              | 0.6573 | 0.5978      | 0.7503       | 0.6655    | 0.5400 | 0.6655 | \          |
| GMM                 | 0.8984 | 0.8983      | 0.9011       | 0.8997    | 0.9039 | 0.8997 | 0.5012     |
| Spectral Clustering | 0.8032 | 0.7960      | 0.8156       | 0.8057    | \      | 0.8057 | \          |

从评测结果来看，我们可以综合地将表现效果分为四个梯队，由于某些算法自身结构的原因，无法完整得出所有的 Silhouette 结果，因此综合分析所有指标后，我们得出以下结论：

- A. 综合来看 GMM 明显优于其他所有算法，GMM 在所有核心指标（AMI、V-measure、ARI、NMI）上均取得最高分，表现最稳定，特别是在 ARI 达到 0.9039，说明其对真实类别结构的恢复程度最好。由于鸢尾花数据中 Versicolor 与 Virginica 存在椭圆形、部分重叠的分布结构，GMM 的多维高斯建模能够自然刻画这种分布，因此获得最佳聚类性能。
- B. 仅次于 GMM 的第二梯队应当是 Spectral Clustering 和 Agglomerative Clustering，Spectral Clustering（V-measure  $\sim 0.806$ ）与 Agglomerative（ $\sim 0.770$ ）表现相近，其中谱聚类稍优。分析原因后，我们认为谱聚类利用相似度图捕捉非线性结构，因此能部分克服 Versicolor 和 Virginica 的边界重叠问题；凝聚层次聚类通过距离矩阵构建层次关系，也能较好地分离 Setosa，并在部分混合区域做出合理划分。
- C. 然后第三梯队则是聚类算法中经典算法——K-Means，K-Means 作为划分式算法表现中等，其各项指标均低于谱聚类和层次聚类，但整体仍保持较稳定的表现（V-measure  $\approx 0.742$ ）。综合来看，K-Means 假设簇为凸球状结构，而鸢尾花的两类（Versicolor 和 Virginica）显然存在椭圆分布与边界交叠，因此其线性分界假设导致无法准确分开这两类。
- D. 最后第四梯队则是 DBSCAN，它的表现最差，主要原因是鸢尾花数据密度较均匀，

不存在明显“密度簇”结构，并且 eps 和 MinPts 难以同时适配三类鸢尾花的微弱密度差异，导致部分簇被拆分或部分样本被定义为噪声点。经过分析我们可以认为 DBSCAN 更适合具有明显密度差异或存在噪声分布的任务，而鸢尾花数据显然不满足。



## 5. 个人体会

通过本次对鸢尾花数据集的聚类实验，我对聚类算法有了比理论学习阶段更加具体、深刻的理解。在实际动手过程中，我不仅验证了不同聚类算法的建模假设和数学特性如何影响最终聚类表现，也意识到数据本身的结构在多大程度上决定了算法的效果。实验结果显示，不同算法之间的差异并非来自单纯的“好”与“坏”，而是来自其面向的数据结构是否匹配。例如，GMM 能够取得最优效果，是因为它利用概率分布建模的方式恰好契合了鸢尾花中椭圆形、存在重叠的类别结构；而 K-Means、层次聚类 and 谱聚类等方法在整体表现上也各有特点，这让我真正理解了“算法适配度”的重要性。另一方面，DBSCAN 的较差表现也让我体会到选择算法必须结合数据特性，不能机械套用所谓“万能算法”。在调参与可视化的过程中，我也进一步认识到评测指标并不能单独用来衡量算法优劣，尤其是当像 Silhouette 这样的几何指标与 AMI、ARI 等基于真实标签的指标产生矛盾时，更需要从数据形态、簇结构及算法假设等多层面综合分析。总的来说，这次实验帮助我建立了对聚类算法更全面的认知，使我能够从算法原理、数据特性、实验结果和评价体系四个维度进行思考，也提升了我将理论知识转化为实践能力的水平，为后续学习更复杂的无监督学习方法奠定了扎实基础。